

Part 1: Developing a Calculator Application Using TDD and OOP

Test-Driven Development (TDD) follows the Red-Green-Refactor cycle:

1. **Red:** Write a failing unit test for the desired behavior.
2. **Green:** Write the simplest code to make the test pass.
3. **Refactor:** Improve the code's design and readability while keeping all tests passing.

We'll build a simple Calculator class step by step.

1. Addition Feature

Step 1: Write a Failing Test (Red)

```
Python
import unittest
class TestCalculator(unittest.TestCase):
    def test_addition(self):
        calc = Calculator() # This will fail - class doesn't exist yet
        result = calc.add(2, 3)
        self.assertEqual(result, 5)
if __name__ == "__main__":
    unittest.main()
```

This test fails with a `NameError` because the `Calculator` class hasn't been defined yet.

Step 2: Make It Pass (Green)

```
python
class Calculator:
    def add(self, a, b):
        return a + b
```

The test now passes.

Step 3: Refactor

The code is already simple and clear—no changes needed yet.

2. Subtraction Feature

Step 1: Write a Failing Test (Red)

Add this method to the test class:

python

```
def test_subtraction(self):  
    calc = Calculator()  
    result = calc.subtract(5, 3)  
    self.assertEqual(result, 2)
```

It fails with an `AttributeError` (no `subtract` method).

Step 2: Make It Pass (Green)

Update the `Calculator` class:

python

```
class Calculator:  
    def add(self, a, b):  
        return a + b  
  
    def subtract(self, a, b):  
        return a - b
```

Step 3: Refactor

We'll address repeated object creation in tests after adding more features.

3. Multiplication Feature

Step 1: Write a Failing Test (Red)

python

```
def test_multiplication(self):
```

```
calc = Calculator()
result = calc.multiply(4, 3)
self.assertEqual(result, 12)
```

Step 2: Make It Pass (Green)

python

```
class Calculator:
    def add(self, a, b):
        return a + b
    def subtract(self, a, b):
        return a - b
    def multiply(self, a, b):
        return a * b
```

Step 3: Refactor the Test Suite

Use `setUp()` to create a fresh `Calculator` instance for each test and eliminate duplication:

python

```
class TestCalculator(unittest.TestCase):
    def setUp(self):
        self.calc = Calculator()

    def test_addition(self):
        self.assertEqual(self.calc.add(2, 3), 5)

    def test_subtraction(self):
        self.assertEqual(self.calc.subtract(5, 3), 2)
```

```

def test_multiplication(self):
    self.assertEqual(self.calc.multiply(4, 3), 12)

if __name__ == "__main__":
    unittest.main()

```

Part 2: Advanced Design Patterns (Refactoring)

To make the calculator more flexible, maintainable, and extensible, we refactor it using established object-oriented design patterns.

1. Strategy Pattern

This pattern lets us define interchangeable algorithms (operations) independently of the Calculator class.

```

python
from abc import ABC, abstractmethod

# Strategy Interface
class OperationStrategy(ABC):
    @abstractmethod
    def execute(self, a: float, b: float) -> float:
        pass

# Concrete Strategies
class AdditionStrategy(OperationStrategy):

```

```

    def execute(self, a, b):
        return a + b

class SubtractionStrategy(OperationStrategy):
    def execute(self, a, b):
        return a - b

class MultiplicationStrategy(OperationStrategy):
    def execute(self, a, b):
        return a * b

# Context
class Calculator:
    def __init__(self, strategy: OperationStrategy = None):
        self._strategy = strategy

    def set_strategy(self, strategy: OperationStrategy):
        self._strategy = strategy

    def calculate(self, a, b):
        if not self._strategy:
            raise ValueError("Strategy not set.")
        return self._strategy.execute(a, b)

```

Usage example:

```

python
calc = Calculator(AdditionStrategy())
print(calc.calculate(2, 3)) # 5

```

```
calc.set_strategy(MultiplicationStrategy())
print(calc.calculate(4, 3))  # 12
```

2. Decorator Pattern

Decorators allow us to add behaviour (e.g., logging) dynamically without modifying the original strategies.

python

```
class LoggingDecorator(OperationStrategy):
    def __init__(self, strategy: OperationStrategy):
        self._strategy = strategy

    def execute(self, a, b):
        print(f"[LOG] Executing operation with inputs: {a}, {b}")
        result = self._strategy.execute(a, b)
        print(f"[LOG] Result: {result}")
        return result
```

Usage:

python

```
decorated = LoggingDecorator(AdditionStrategy())
calc = Calculator(decorated)
calc.calculate(5, 3)
```

3. Visitor Pattern (for Analytics/Reporting)

This separates reporting logic from the main classes.

python

```
from abc import ABC, abstractmethod
```

```
class Visitor(ABC):
```

```
    @abstractmethod
```

```
    def visit_calculator(self, calculator):
```

```
        pass
```

```
class AnalyticsVisitor(Visitor):
```

```
    def __init__(self):
```

```
        self.execution_count = 0
```

```
    def visit_calculator(self, calculator):
```

```
        self.execution_count += 1
```

```
        print(f"[ANALYTICS] Operations triggered: {self.execution_count}")
```

```
# Add to Calculator class:
```

```
# def accept(self, visitor: Visitor):
```

```
#     visitor.visit_calculator(self)
```

4. Abstract Factory Pattern

Useful for creating families of related objects (e.g., local vs. remote calculators in a larger system).

python

```

class CalculatorFactory(ABC):
    @abstractmethod
    def create_addition(self) -> OperationStrategy:
        pass

    @abstractmethod
    def create_subtraction(self) -> OperationStrategy:
        pass


class LocalEngineFactory(CalculatorFactory):
    def create_addition(self):
        return AdditionStrategy()

    def create_subtraction(self):
        return SubtractionStrategy()

```

Part 3: Testing with Mocking and Dependency Injection

We use `unittest.mock` for isolated testing.

```

python

from unittest.mock import Mock


# Example service (e.g., for future extensions like validation)
class ValidationService:
    def validate_inputs(self, a, b):
        if not isinstance(a, (int, float)) or not isinstance(b,
(int, float)):
            raise ValueError("Inputs must be numbers")

        return True

```



```

class Calculator:
    def __init__(self, strategy: OperationStrategy, validator=None):
        self._strategy = strategy
        self._validator = validator or ValidationService()

    def calculate(self, a, b):
        self._validator.validate_inputs(a, b)
        return self._strategy.execute(a, b)

```

Test with Mock:

python

```

class TestCalculator(unittest.TestCase):
    def test_with_mocked_validator(self):
        mock_validator = Mock()
        mock_validator.validate_inputs.return_value = True

        calc = Calculator(AdditionStrategy(), mock_validator)
        result = calc.calculate(2, 3)

        self.assertEqual(result, 5)
        mock_validator.validate_inputs.assert_called_once_with(2, 3)

```

References

Beck, K. (2003). Test-Driven Development: By Example. Addison-Wesley Professional.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional.

Fowler, M. (2018). Refactoring: Improving the Design of Existing Code. Addison-Wesley Professional.